

Dynace

ODBC User Manual
April 6, 2026

by Blake McBride

Copyright © 1996 Blake McBride All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Windows and Microsoft are registered trademarks of Microsoft Corporation. T_EX is a trademark of the American Mathematical Society. Other brand and product names are trademarks or registered trademarks of their respective holders.

This manual was typeset with the T_EX typesetting system developed by Donald Knuth.

1 ODBC Tutorial

1.1 Opening and Closing a Database

Prior to any use of a database, it must be opened. Two functions are available to accommodate the function as follows:

`OpenDatabase::Database` [OpenDatabase]

```
DB = gOpenDatabase(Database, source, id, pw)

object DB;           /* database object */
char *source;        /* data source name as defined with
                      the ODBC administrator */
char *id;            /* the user id to log into the
                      database with */
char *pw;            /* the password associated with
                      the user id */
```

This will open the specified database using the specified user id and password. NULL is returned if the database cannot be opened.

`QueryDatabase::Database` [QueryDatabase]

```
DB = gQueryDatabase(Database, wind)

object DB;           /* database object */
object wind;         /* parent window object or NULL */
```

This will query the user to select a database and then open the specified database using the user ID and password entered using the ones configured with the ODBC administrator.

The following method is used to close a database.

`Dispose::Database` [Dispose]

```
DB = gDispose(DB)

object DB;           /* database object */
```

This procedure will close the database and NULL out the DB variable.

1.2 Statements

1.2.1 Introduction

One concept which is important to fully understand is the *Statement* object. A Statement object allows the program a means to execute an SQL command and then have access to the data returned by that particular command. Once a command's data is no longer needed, the Statement object may be re-used for other (and arbitrary) SQL commands. A single database may have numerous Statement objects associated with it at any given point. This number is only limited by the particular database vendor.

1.2.2 Creating and Disposing of Statement Objects

`NewStatement::Database` [NewStatement]

```
stmt = gNewStatement(DB);

object DB;      /* the database object */
object stmt;    /* a new statement object */
```

This will create a new statement object;

`Dispose::Statement` [Dispose]

```
gDispose(stmt);

object stmt;    /* the statement object */
```

This will dispose of the statement object;

1.3 Adding Records

1.3.1 Overview

Adding of records is accomplished in three steps as follows.

1. Select the file or table in which records are to be added. This only needs to be done once even if multiple records are to be added.
2. Assign the data to the fields.
3. Add the record.

Once a record is added, additional records may be added by looping through steps 2 and 3. Data assigned to a field which is invariant need not be re-assigned.

1.3.2 Selecting the Table

Selecting the table can be accomplished in two ways. The most common way would be via the `gInsert` method. Alternatively, if you are already in the middle of a select command in which all the fields are selected in the desired table and there are no joins, as follows:

```
gDBSelect(stmt, "select * from mytable");
```

you need not perform any additional function. The table is already selected.

Insert::Statement**[Insert]**

```

    r = gInsert(stmt, table);

    object  stmt;  /* the statement object */
    char    *table; /* the table to use      */
    int     r;     /* 0 on success          */

```

This method selects a table for subsequent record adds.

1.3.3 Assigning Field Data

Once the desired table is selected, field data may be assigned using the following methods.

```

    stmt = gFldSetString(stmt, fld, str);

    stmt = gFldSetChar(stmt, fld, ival);

    stmt = gFldSetShort(stmt, fld, ival);

    stmt = gFldSetUnsignedShort(stmt, fld, uival);

    stmt = gFldSetLong(stmt, fld, lval);

    stmt = gFldSetDouble(stmt, fld, dval);

    object  stmt;  /* the statement to operate on */
    char    *fld;  /* the field name           */
    int     ival;  /* an integer value         */
    unsigned int uival; /* unsigned value          */
    long    lval;  /* long value               */
    double  dval;  /* double value             */

```

The object returned is simply the statement object passed. Any attempt to use a non-existing field will cause an error in which the application will be terminated. If you assign a value of the wrong type, the system will convert it for you. Date fields are simply converted to `long`'s of the form `YYYYMMDD`.

1.3.4 Adding The Record

Once the data has been assigned the record may be added as follows.

AddRecord::Statement**[AddRecord]**

```

r = gAddRecord(stmt);

object stmt; /* the statement object */
int     r;    /* result                */

```

This will cause the data to be added to the table. Zero is returned upon success, other values indicate an error. Most errors are trapped and cause the application to abort, however.

1.4 Reading Records

1.4.1 Overview

Reading records is accomplished in three steps as follows.

1. Select a set of records from which to access data.
2. Select a particular record.
3. Access the field data.

1.4.2 Selecting a Record Set

Selecting a record set is accomplished by executing a normal SQL `select` command as follows.

DBSelect::Statement**[DBSelect]**

```

r = gDBSelect(stmt, cmd);

object stmt; /* the statement object */
char    *cmd; /* the SQL select command */
int     r;    /* result                */

```

This method will execute an arbitrary SQL select command given in `cmd`. This command may contain joins, order by, and where clauses as well as any other legal SQL select command. zero is returned upon success. For example:

```
gDBSelect(stmt, "select * from myfile");
```

DBSelectOne::Statement**[DBSelectOne]**

```

r = gDBSelectOne(stmt, cmd);

object stmt; /* the statement object */
char    *cmd; /* the SQL select command */
int     r;    /* result                */

```

This method is a convenience for the case when a single specific record is desired. When this method is used there is no need to also select a specific record. Data fields may be accessed immediately. zero is returned upon success.

1.4.3 Selecting a Record

The following functions allow the selection of a specific record within a selected set. Note that if `gDBSelectOne` is used, none of these functions should be used.

```
r = gFirstRecord(stmt);

r = gLastRecord(stmt);

r = gNextRecord(stmt);

r = gPrevRecord(stmt);

r = gSetAbsPos(stmt, pos);

r = gSetRelPos(stmt, pos);

object stmt; /* the statement object */
long pos; /* relative or absolute position */
int r; /* 0=success */
```

1.4.4 Accessing Field Data

The following functions are used to access field data.

```

sval = gFldGetString(stmt, fld);

cval = gFldGetChar(stmt, fld);

sval = gFldGetShort(stmt, fld);

uval = gFldGetUnsignedShort(stmt, fld);

lval = gFldGetLong(stmt, fld);

dval = gFldGetDouble(stmt, fld);

oval = gFldGetValue(stmt, fld);

object stmt; /* the statement object */
char    *fld; /* the field name          */
short   sval; /* return values           */
char    cval;
unsigned short uval;
long     lval;
double   dval;
object   oval;

```

If the field type is not the same as that requested, the system will convert it for you.

1.5 Changing and Deleting Records

The procedure for changing or deleting records is similar to that of reading records. Steps 1 and 2 are the same. In addition, once a record had been read it may be immediately changed or deleted without the need to re-execute the first two steps. Step 3 is as follows.

UpdateRecord::Statement

[UpdateRecord]

```

r = gUpdateRecord(stmt);
    or
r = gDeleteRecord(stmt);

object stmt; /* the statement object */
int      r;   /* result (0=success)   */

```

This method will either update or delete the last selected record.

1.6 Executing SQL Commands

The following methods allow the execution of arbitrary SQL commands.

Execute[\[Execute\]](#)

```

r = gExecute(DB, cmd);
    or
r = gExecute(stmt, cmd);

object DB;      /* database object */
object stmt;    /* statement object */
char *cmd;      /* SQL command */
int r;          /* result (0=success) */

```

These methods allow for the execution of arbitrary SQL command when no specific data access is required (you aren't trying to access field values).

ExecuteFile[\[ExecuteFile\]](#)

```

r = gExecuteFile(DB, file);
    or
r = gExecuteFile(stmt, file);

object DB;      /* database object */
object stmt;    /* statement object */
char *file;     /* file name */
int r;          /* result (0=success) */

```

The method will cause all the SQL commands in a file to be executed against database DB. A file extension of `.sql` is assumed.

1.7 Dropping Tables

DropTable::Database[\[DropTable\]](#)

```

r = gDropTable(DB, tbl);

object DB;      /* database object */
char *tbl;      /* table */
int r;          /* result (0=success) */

```

The method will drop table `tbl`. If the table didn't already exist this command will be ignored.

1.8 Clearing Fields

Clear::Statement

[Clear]

```

r = gClear(stmt);

object stmt; /* statement object */
int     r;    /* result (0=success) */

```

This method will clear (zero out) all the fields. It may only be used after a select command. It is often used after some records have been read and a new record is to be added without an intervening record set selection.

1.9 WDS Interface

The following methods are used to associate ODBC data fields to dialog controls.

AttachDialog::Statement

[AttachDialog]

```

stmt = gAttachDialog(stmt, dlg);

object stmt; /* statement object */
object dlg;  /* a dialog object */

```

This method is used to associate all controls within a dialog to fields associated with a statement in which the fields and controls have the same name. Once this association is formed subsequent use of the dialog will cause the data to be entered into the associated data fields. This method should only be used subsequent to a `gDBSelect`, `gDBSelectOne`, or a `gInsert`.

AssociateCtl::Statement

[AssociateCtl]

```

stmt = gAssociateCtl(stmt, fld, ctl);

object stmt; /* statement object */
char   *fld; /* a database field */
object ctl;  /* a control object */

```

This method is used to associate control `ctl` with database field `fld`. Once this association is formed subsequent use of the control will cause the data to be entered into the associated data field. This method should only be used subsequent to a `gDBSelect`, `gDBSelectOne`, or a `gInsert`.

1.10 Error Handling

The following methods provide access to error information when a database or statement operation fails.

GetErrorMessage::Database

[GetErrorMessage]

```
s = gGetErrorMessage(DB);
```

```
object DB;    /* database object    */
char    *s;    /* error message or NULL    */
```

Returns the error message string from the most recent database operation, or NULL if no error occurred.

GetErrorMessage::Statement

[GetErrorMessage]

```
s = gGetErrorMessage(stmt);
```

```
object stmt; /* statement object    */
char    *s;    /* error message or NULL    */
```

Returns the error message string from the most recent statement operation, or NULL if no error occurred.

GetErrorCode::Database

[GetErrorCode]

```
code = gGetErrorCode(DB);
```

```
object DB;    /* database object    */
int    code; /* error code        */
```

Returns the error code from the most recent database operation.

GetErrorCode::Statement

[GetErrorCode]

```
code = gGetErrorCode(stmt);
```

```
object stmt; /* statement object    */
int    code; /* error code        */
```

Returns the error code from the most recent statement operation.

GetErrorState::Statement

[GetErrorState]

```
s = gGetErrorState(stmt);
```

```
object stmt; /* statement object    */
char    *s;    /* ODBC SQLSTATE code    */
```

Returns the ODBC SQLSTATE error code string from the most recent statement operation.

GetNativeErrorCode::Statement

[GetNativeErrorCode]

```
code = gGetNativeErrorCode(stmt);

object stmt; /* statement object */
int     code; /* native error code */
```

Returns the native database-specific error code from the most recent statement operation.

1.11 Extended Field Operations

In addition to the basic field get and set methods described earlier, the following methods provide access to additional field types and operations.

FldGetDateTime::Statement

[FldGetDateTime]

```
stmt = gFldGetDateTime(stmt, fld, dt, tm);

object stmt; /* statement object */
char    *fld; /* field name */
long    *dt;  /* pointer to date (YYYYMMDD) */
long    *tm;  /* pointer to time (HHMMSS) */
```

Retrieves a date/time field as two separate long values: a date in YYYYMMDD format and a time in HHMMSS format.

See also: **FldSetDateTime**

FldGetPointer::Statement

[FldGetPointer]

```
p = gFldGetPointer(stmt, fld);

object stmt; /* statement object */
char    *fld; /* field name */
void    *p;   /* pointer value */
```

Returns a pointer to the internal field data.

FldGetStringNoStrip::Statement**[FldGetStringNoStrip]**

```

s = gFldGetStringNoStrip(stmt, fld);

object stmt; /* statement object */
char   *fld; /* field name       */
char   *s;   /* string value     */

```

Returns the field value as a string without stripping trailing whitespace. By contrast, `gFldGetString` trims trailing spaces.

See also: `FldGetString`

FldSetDateTime::Statement**[FldSetDateTime]**

```

stmt = gFldSetDateTime(stmt, fld, dt, tm);

object stmt; /* statement object */
char   *fld; /* field name       */
long   dt;   /* date value (YYYYMMDD) */
long   tm;   /* time value (HHMMSS)  */

```

Sets a date/time field from separate date and time long values.

See also: `FldGetDateTime`

FldSetNull::Statement**[FldSetNull]**

```

stmt = gFldSetNull(stmt, fld);

object stmt; /* statement object */
char   *fld; /* field name       */

```

Sets the specified field to SQL NULL.

See also: `FldSetNotNull`

FldSetNotNull::Statement**[FldSetNotNull]**

```

stmt = gFldSetNotNull(stmt, fld);

object stmt; /* statement object */
char   *fld; /* field name       */

```

Marks the specified field as not NULL. This is useful after calling `gFldSetNull` to restore the field to a non-null state before assigning a new value.

See also: `FldSetNull`

`FldToFile::Statement`

[`FldToFile`]

```

r = gFldToFile(stmt, fld, file);

object stmt; /* statement object */
char   *fld; /* field name       */
char   *file; /* output file path */
int    r;     /* result           */

```

Writes the contents of the specified field to a file. Useful for extracting binary or large text fields.

See also: `FldSetFromFile`

`FldSetFromFile::Statement`

[`FldSetFromFile`]

```

stmt = gFldSetFromFile(stmt, fld, file);

object stmt; /* statement object */
char   *fld; /* field name       */
char   *file; /* input file path  */

```

Sets the specified field's contents from a file. Useful for storing binary or large text data.

See also: `FldToFile`

`FldSetBinary::Statement`

[`FldSetBinary`]

```

stmt = gFldSetBinary(stmt, fld, size, val);

object stmt; /* statement object */
char   *fld; /* field name       */
long    size; /* data size in bytes */
char   *val;  /* pointer to binary data */

```

Sets a binary field from a memory buffer of the specified size.

See also: `FldGetBinary`

FldGetBinary::Statement**[FldGetBinary]**

```

    stmt = gFldGetBinary(stmt, fld, size, val);

    object  stmt;    /* statement object          */
    char    *fld;    /* field name                */
    long    *size;   /* pointer to receive data size */
    char    **val;   /* pointer to receive data pointer */

```

Retrieves binary field data. The size and a pointer to the data are returned through the output parameters.

See also: **FldSetBinary**

1.12 Additional Record Operations

RecordCount::Statement**[RecordCount]**

```

    n = gRecordCount(stmt);

    object  stmt;    /* statement object          */
    int     n;       /* number of records        */

```

Returns the number of records in the current result set.

RereadRecord::Statement**[RereadRecord]**

```

    r = gRereadRecord(stmt);

    object  stmt;    /* statement object          */
    int     r;       /* result (0=success)       */

```

Re-reads the current record from the database, refreshing all field values. Useful when the record may have been changed by another user or process.

UnattachDialog::Statement**[UnattachDialog]**

```

    stmt = gUnattachDialog(stmt, dlg);

    object  stmt;    /* statement object          */
    object  dlg;     /* dialog object             */

```

Removes the association between a statement's fields and a dialog's controls. This is the inverse of **gAttachDialog**.

See also: `AttachDialog`

`CopyCorresponding::Statement`

[`CopyCorresponding`]

```
stmt = gCopyCorresponding(stmt, from);

object stmt; /* destination statement */
object from; /* source statement      */
```

Copies field values from the source statement to the destination statement for all fields that have matching names. Useful for copying data between different tables that share common fields.

1.13 Database Properties

`GetCurrentDataSource::Database`

[`GetCurrentDataSource`]

```
DB = gGetCurrentDataSource(DB, buf);

object DB; /* database object */
char *buf; /* buffer to receive name */
```

Copies the current data source name into `buf`.

`GetCurrentDatabaseName::Database`

[`GetCurrentDatabaseName`]

```
DB = gGetCurrentDatabaseName(DB, buf);

object DB; /* database object */
char *buf; /* buffer to receive name */
```

Copies the current database name into `buf`.

`DBMS_type::Database`

[`DBMS_type`]

```
s = gDBMS_type(DB);

object DB; /* database object */
char *s; /* DBMS type string */
```

Returns a string identifying the database management system type (e.g. “Microsoft SQL Server”, “MySQL”, “PostgreSQL”).

DBMS_version::Database

[DBMS_version]

```

s = gDBMS_version(DB);

object DB;    /* database object      */
char   *s;    /* DBMS version string    */

```

Returns a string containing the DBMS version number.

GetUserName::Database

[GetUserName]

```

s = gGetUserName(DB);

object DB;    /* database object      */
char   *s;    /* user name string     */

```

Returns the user name used for the current database connection.

1.14 Vararg SQL Methods

The following methods accept `printf`-style format strings, allowing SQL commands to be built inline with variable substitution.

vDBSelect::Statement

[vDBSelect]

```

r = vDBSelect(stmt, fmt, ...);

object stmt;    /* statement object      */
char   *fmt;    /* printf-style format string */
int     r;      /* result (0=success)      */

```

Executes an SQL `SELECT` using a `printf`-style format string. For example:

```
vDBSelect(stmt, "select * from emp where id = %ld", empId);
```

See also: `DBSelect`, `vDBSelectOne`

vDBSelectOne::Statement

[vDBSelectOne]

```

r = vDBSelectOne(stmt, fmt, ...);

object stmt;    /* statement object      */
char   *fmt;    /* printf-style format string */
int     r;      /* result (0=success)      */

```

Convenience method that combines `vDBSelect` with an implicit `gFirstRecord`. Use when exactly one record is expected.

See also: DBSelectOne, vDBSelect

vExecute::Statement

[vExecute]

```
r = vExecute(stmt, fmt, ...);

object stmt; /* statement object */
char *fmt; /* printf-style format string */
int r; /* result (0=success) */
```

Executes an arbitrary SQL command using a `printf`-style format string. This is the vararg form of `gExecute`.

```
vExecute(stmt, "delete from emp where id = %ld", empId);
```

See also: `Execute`

1.15 Schema Introspection

The following methods allow inspection of database structure and metadata.

GetAllTables::Database

[GetAllTables]

```
r = gGetAllTables(DB);

object DB; /* database object */
int r; /* result */
```

Retrieves metadata for all tables in the database.

SelectColumns::Statement

[SelectColumns]

```
r = gSelectColumns(stmt, table, owner);

object stmt; /* statement object */
char *table; /* table name */
char *owner; /* owner/schema or NULL */
int r; /* result (0=success) */
```

Retrieves column metadata for the specified table. After calling this method, use `gNextRecord` and the field access methods to iterate through the column definitions.

SelectTables::Statement

[SelectTables]

```

r = gSelectTables(stmt, owner);

object stmt;    /* statement object      */
char   *owner;  /* owner/schema or NULL  */
int     r;      /* result (0=success)    */

```

Retrieves a list of all tables in the database (optionally filtered by owner). Use `gNextRecord` to iterate through the results.

SelectPrimaryKeyFields::Statement

[SelectPrimaryKeyFields]

```

r = gSelectPrimaryKeyFields(stmt, table, owner);

object stmt;    /* statement object      */
char   *table;  /* table name            */
char   *owner;  /* owner/schema or NULL  */
int     r;      /* result (0=success)    */

```

Retrieves the primary key columns for the specified table.

SelectForeignKeys::Statement

[SelectForeignKeys]

```

r = gSelectForeignKeys(stmt, table, owner);

object stmt;    /* statement object      */
char   *table;  /* table name            */
char   *owner;  /* owner/schema or NULL  */
int     r;      /* result (0=success)    */

```

Retrieves the foreign key relationships for the specified table.

SelectIndexes::Statement

[SelectIndexes]

```

r = gSelectIndexes(stmt, table, owner);

object stmt;    /* statement object      */
char   *table;  /* table name            */
char   *owner;  /* owner/schema or NULL  */
int     r;      /* result (0=success)    */

```

Retrieves index information for the specified table.

1.16 Data Change Tracking

`DataChanged::Database` [DataChanged]

```
val = gDataChanged(DB);

object DB;    /* database object    */
int      val; /* 1=changed, 0=unchanged */
```

Returns 1 if data has been changed since the last call to `gResetChanged`, 0 otherwise.

See also: `SetChanged`, `ResetChanged`

`ResetChanged::Database` [ResetChanged]

```
r = gResetChanged(DB);

object DB; /* database object */
```

Resets the data-changed flag to 0.

See also: `DataChanged`, `SetChanged`

`SetChanged::Database` [SetChanged]

```
r = gSetChanged(DB);

object DB; /* database object */
```

Manually sets the data-changed flag to 1.

See also: `DataChanged`, `ResetChanged`

1.17 Write Control

`DisableWrites::Database` [DisableWrites]

```
r = gDisableWrites(DB);

object DB; /* database object */
```

Disables all write operations (insert, update, delete) on the database. Attempts to write will be silently ignored.

See also: `EnableWrites`

`EnableWrites::Database`

[`EnableWrites`]

```
r = gEnableWrites(DB);
```

```
object DB; /* database object */
```

Re-enables write operations after a call to `gDisableWrites`.

See also: `DisableWrites`

1.18 Error Handling Modes

`IgnoreAllErrors::Statement`

[`IgnoreAllErrors`]

```
prev = gIgnoreAllErrors(stmt, val);
```

```
object stmt; /* statement object */
```

```
int val; /* 1=ignore, 0=normal */
```

```
int prev; /* previous setting */
```

When enabled, all SQL errors are silently ignored instead of aborting the application. Returns the previous setting.

See also: `ReturnAllErrors`

`ReturnAllErrors::Statement`

[`ReturnAllErrors`]

```
prev = gReturnAllErrors(stmt, val);
```

```
object stmt; /* statement object */
```

```
int val; /* 1=return errors, 0=normal */
```

```
int prev; /* previous setting */
```

When enabled, SQL errors are returned as error codes instead of aborting the application. This allows the caller to handle errors programmatically. Returns the previous setting.

See also: `IgnoreAllErrors`

Method Index

A

AddRecord::Statement	4
AssociateCtl::Statement	8
AttachDialog::Statement	8

C

Clear::Statement	8
CopyCorresponding::Statement	14

D

DataChanged::Database	18
DBMS_type::Database	14
DBMS_version::Database	15
DBSelect::Statement	4
DBSelectOne::Statement	4
DisableWrites::Database	18
Dispose::Database	1
Dispose::Statement	2
DropTable::Database	7

E

EnableWrites::Database	19
Execute	7
ExecuteFile	7

F

FldGetBinary::Statement	13
FldGetDateTime::Statement	10
FldGetPointer::Statement	10
FldGetStringNoStrip::Statement	11
FldSetBinary::Statement	12
FldSetDateTime::Statement	11
FldSetFromFile::Statement	12
FldSetNotNull::Statement	11
FldSetNull::Statement	11
FldToFile::Statement	12

G

gCopyCorresponding	14
gDataChanged	18
gDBMS_type	14
gDBMS_version	15
gDisableWrites	18
gEnableWrites	19
GetAllTables::Database	16
GetCurrentDatabaseName::Database	14
GetCurrentDataSource::Database	14
GetErrorCode::Database	9
GetErrorCode::Statement	9
GetErrorMessage::Database	9
GetErrorMessage::Statement	9
GetErrorState::Statement	9
GetNativeErrorCode::Statement	10
GetUserName::Database	15
gFldGetBinary	13
gFldGetDateTime	10
gFldGetPointer	10
gFldGetStringNoStrip	11
gFldSetBinary	12
gFldSetDateTime	11
gFldSetFromFile	12
gFldSetNotNull	11
gFldSetNull	11
gFldToFile	12
gGetAllTables	16
gGetCurrentDatabaseName	14
gGetCurrentDataSource	14
gGetErrorCode::Database	9
gGetErrorCode::Statement	9
gGetErrorMessage::Database	9
gGetErrorMessage::Statement	9
gGetErrorState	9
gGetNativeErrorCode	10
gGetUserName	15
gIgnoreAllErrors	19
gRecordCount	13
gRereadRecord	13
gResetChanged	18
gReturnAllErrors	19
gSelectColumns	16
gSelectForeignKeys	17
gSelectIndexes	17
gSelectPrimaryKeyFields	17
gSelectTables	17
gSetChanged	18
gUnattachDialog	13

I

IgnoreAllErrors::Statement..... 19
 Insert::Statement..... 3

N

NewStatement::Database..... 2

O

OpenDatabase::Database..... 1

Q

QueryDatabase::Database..... 1

R

RecordCount::Statement..... 13
 RereadRecord::Statement..... 13
 ResetChanged::Database..... 18
 ReturnAllErrors::Statement..... 19

S

SelectColumns::Statement..... 16
 SelectForeignKeys::Statement..... 17
 SelectIndexes::Statement..... 17
 SelectPrimaryKeyFields::Statement..... 17
 SelectTables::Statement..... 17
 SetChanged::Database..... 18

U

UnattachDialog::Statement..... 13
 UpdateRecord::Statement..... 6

V

vDBSelect..... 15
 vDBSelect::Statement..... 15
 vDBSelectOne..... 15
 vDBSelectOne::Statement..... 15
 vExecute..... 16
 vExecute::Statement..... 16

Table of Contents

1	ODBC Tutorial	1
1.1	Opening and Closing a Database	1
1.2	Statements	1
1.2.1	Introduction	2
1.2.2	Creating and Disposing of Statement Objects	2
1.3	Adding Records	2
1.3.1	Overview	2
1.3.2	Selecting the Table	2
1.3.3	Assigning Field Data	3
1.3.4	Adding The Record	3
1.4	Reading Records	4
1.4.1	Overview	4
1.4.2	Selecting a Record Set	4
1.4.3	Selecting a Record	5
1.4.4	Accessing Field Data	5
1.5	Changing and Deleting Records	6
1.6	Executing SQL Commands	6
1.7	Dropping Tables	7
1.8	Clearing Fields	8
1.9	WDS Interface	8
1.10	Error Handling	8
1.11	Extended Field Operations	10
1.12	Additional Record Operations	13
1.13	Database Properties	14
1.14	Vararg SQL Methods	15
1.15	Schema Introspection	16
1.16	Data Change Tracking	18
1.17	Write Control	18
1.18	Error Handling Modes	19
	Method Index	21

